

手势识别实验指南

一、实验概述

1.1 实验背景

手势识别是物联网应用软件开发中的典型智能感知任务。摄像头或图片作为感知输入，边缘端程序完成手部检测、关键点提取、手势分类和结果输出，最后可以进一步驱动灯光、门禁、屏幕控制、智能家居或人机交互系统。

本实验基于开源项目 `hand-gesture-recognition-using-mediapipe`，使用 MediaPipe 提取 21 个手部关键点，再使用 TensorFlow Lite 模型识别手势。原项目默认使用摄像头实时识别；考虑到部分电脑没有摄像头，本实验已增加图片输入模式，学生可以直接对图片进行识别并保存标注结果。

1.2 实验目标

完成本实验后，学生应能够：

- 理解“图像采集 -> 手部检测 -> 关键点提取 -> 特征归一化 -> 手势分类 -> 应用输出”的完整流程。
- 掌握 Python 视觉项目的依赖安装、运行和常见错误排查方法。
- 能够使用命令行对单张图片进行手势识别并生成结果图。
- 能够修改 OpenCV 标注代码，解决文字被裁切、不可见等实际工程问题。
- 能够设计一个简单的物联网手势控制应用逻辑。
- 能够完成若干小型拓展实验，使项目从“能运行”变成“能分析、能改造、能应用”。

1.3 项目功能介绍

本项目包含两类识别功能：

类型	功能	当前标签
静态手势识别	根据单帧手部关键点识别手势	Open、Close、Pointer、OK
动态手势识别	根据食指指尖历史轨迹识别动作	Stop、Clockwise、Counter Clockwise、Move

本次课堂重点使用静态手势识别。动态手势识别需要连续视频帧或摄像头输入，可作为拓展讨论。

1.4 物联网应用场景

场景	应用描述
智能家居	张开手掌开灯，握拳关灯，OK 手势确认操作
智能展厅	游客通过手势控制屏幕翻页、选择展品
无接触设备控制	在医院、工厂等场景下减少实体按钮接触
边缘 AI 网关	摄像头采集画面，边缘设备本地完成识别，只上传识别结果
辅助教学	识别学生举手、确认、指向等课堂动作

二、实验环境

2.1 推荐环境

项目	推荐配置
操作系统	Windows 10/11
Python	Python 3.10 - 3.12
内存	8GB 及以上
摄像头	可选，本实验支持图片输入
主要依赖	OpenCV、MediaPipe、TensorFlow

2.2 本机验证过的依赖版本

在本实验调试过程中，原 README 中的依赖版本较旧，直接使用最新版依赖会遇到兼容问题。当前已验证可运行的组合如下：

```
python --version
# Python 3.12.10

python -c "import cv2, mediapipe as mp, tensorflow as tf, numpy as np;
print(cv2.__version__); print(mp.__version__); print(tf.__version__);
print(np.__version__)"
# OpenCV: 4.11.0 或兼容版本
# MediaPipe: 0.10.21
# TensorFlow: 2.18.1
# NumPy: 1.26.4
```

推荐安装命令：

```
pip install mediapipe==0.10.21 tensorflow==2.18.1
```

如果 OpenCV 缺失，可执行：

```
pip install opencv-contrib-python==4.11.0.86
```

2.3 目录结构

```
hand-gesture-recognition/
├─ app.py
├─ hand.jpg
├─ result.jpg
├─ keypoint_classification.ipynb
├─ point_history_classification.ipynb
├─ model/
│   └─ keypoint_classifier/
│       ├── keypoint_classifier.tflite
│       ├── keypoint_classifier_label.csv
│       └─ keypoint_classifier.py
│   └─ point_history_classifier/
│       ├── point_history_classifier.tflite
│       ├── point_history_classifier_label.csv
│       └─ point_history_classifier.py
└─ utils/
    └─ cvfpscalc.py
```

核心文件说明：

文件	作用
app.py	主程序入口，负责读取图像/摄像头、调用MediaPipe、分类和绘图
model/keypoint_classifier/keypoint_classifier.tflite	静态手势识别模型
model/keypoint_classifier/keypoint_classifier_label.csv	静态手势标签
keypoint_classification.ipynb	静态手势模型训练 Notebook
point_history_classification.ipynb	动态手势模型训练 Notebook

三、基础实验：运行图片手势识别

3.1 原项目摄像头模式

原项目默认使用摄像头：

```
python app.py
```

如果电脑没有摄像头，程序无法采集图像。因此本实验已在 `app.py` 中增加图片输入参数。

3.2 图片输入模式

将待识别图片放到项目目录，例如：

```
hand.jpg
```

执行：

```
python app.py --image hand.jpg --output result.jpg --no_display
```

参数说明：

参数	说明
<code>--image</code>	输入图片路径
<code>--output</code>	输出结果图片路径
<code>--no_display</code>	不弹出 OpenCV 窗口，只保存图片

运行成功时，终端会输出类似内容：

```
Detected hand signs: Open  
Saved result: result.jpg
```

3.3 查看结果



打开 `result.jpg`，应能看到：

- 手部关键点。
- 手部骨架连接线。
- 手部外接框。
- 手势识别标签，例如 `Right:Open` 或 `Left:Open`。



如果没有识别到手，会输出：

No hand detected.
Saved result: result.jpg

四、核心原理讲解

4.1 系统处理流程

```
graph TD
    A[输入图片/摄像头] --> B[OpenCV 读取图像]
    B --> C[MediaPipe Hands 检测手部]
    C --> D[提取 21 个手部关键点]
    D --> E[关键点坐标相对化和归一化]
    E --> F[TFLite 静态手势分类器]
    F --> G[OpenCV 绘制骨架、边界框、标签]
    G --> H[保存 result.jpg 或显示窗口]
```

4.2 MediaPipe 手部关键点

MediaPipe Hands 会输出 21 个手部关键点，包括手腕、拇指、食指、中指、无名指和小指的关节点。

项目中的关键函数：

函数	作用
<code>calc_landmark_list()</code>	将 MediaPipe 的归一化坐标转换为图片像素坐标
<code>pre_process_landmark()</code>	将绝对坐标转换为相对坐标，并做归一化
<code>KeypointClassifier()</code>	加载 TFLite 静态手势模型并推理
<code>draw_landmarks()</code>	绘制手部骨架和关键点
<code>draw_info_text()</code>	绘制识别标签

4.3 为什么要做坐标归一化

同一个手势可能出现在图片的不同位置、不同大小。如果直接使用像素坐标，模型会受到图片尺寸和手掌位置影响。

项目采用的处理方式：

1. 以第 0 个点，即手腕位置，作为基准点。
2. 所有关键点坐标减去手腕坐标，变成相对坐标。
3. 找到最大绝对值，用它对所有坐标进行缩放。
4. 得到长度固定的 42 维输入特征，即 21 个点的 x/y 坐标。

这使模型更关注“手指之间的相对形状”，而不是手在图片中的绝对位置。

4.4 静态手势模型

静态手势分类器位于：

```
model/keypoint_classifier/keypoint_classifier.tflite
```

标签位于：

```
model/keypoint_classifier/keypoint_classifier_label.csv
```

当前标签：

```
Open  
Close  
Pointer  
OK
```

五、调试记录与常见问题

5.1 问题 1：缺少 MediaPipe

报错：

```
ModuleNotFoundError: No module named 'mediapipe'
```

原因：当前 Python 环境没有安装 `mediapipe`。

解决：

```
pip install mediapipe==0.10.21
```

5.2 问题 2：新版 MediaPipe 不支持旧接口

报错：

```
AttributeError: module 'mediapipe' has no attribute 'solutions'
```

原因：安装了较新的 `mediapipe 0.10.35`，该版本主入口只暴露了 `tasks`，原项目使用的 `mp.solutions.hands` 接口不可用。

解决：降级到兼容旧接口的版本。

```
pip install mediapipe==0.10.21
```

验证：

```
python -c "import mediapipe as mp; print(mp.__version__); print(hasattr(mp, 'solutions'))"
```

应输出：

```
0.10.21
True
```

5.3 问题 3: TensorFlow 与 protobuf 冲突

报错示例：

```
ImportError: cannot import name 'runtime_version' from 'google.protobuf'
```

原因：tensorflow 2.21 需要较新的 protobuf，而 mediapipe 0.10.21 依赖较旧的 protobuf，二者冲突。

解决：使用与 mediapipe 0.10.21 更兼容的 TensorFlow 版本。

```
pip install tensorflow==2.18.1
```

5.4 问题 4: 没有摄像头

现象：运行原命令 `python app.py` 后无法采集图像，或者窗口没有有效画面。

解决：使用本实验改造后的图片输入模式。

```
python app.py --image hand.jpg --output result.jpg --no_display
```

5.5 问题 5: 结果图片中标签看不见

原因：原代码把文字标签固定画在手部边界框上方。如果手指接近图片上边缘，标签会被画到图片外。

修改思路：

- 先判断边界框上方是否有足够空间。
- 如果上方空间不足，就把标签画到边界框下方。
- 使用 `cv.getTextSize()` 根据文字宽度动态计算黑色背景框。

涉及函数：

```
draw_info_text()
```

六、拓展实验

小实验 1：图片输入功能分析

任务目标：理解如何将原摄像头程序改造成图片识别程序。

观察 `app.py` 中新增的参数：

```
parser.add_argument("--image", help='input image path', type=str)
parser.add_argument("--output", help='output image path', type=str)
parser.add_argument("--no_display", action='store_true')
```

思考问题：

1. 摄像头模式和图片模式的数据来源有什么区别？
2. 为什么图片模式中 `static_image_mode` 应该设为 `True`？
3. 为什么中文路径下读写图片时，使用 `cv.imdecode()` 和 `cv.imencode()` 比直接 `cv.imread()` 更稳妥？

小实验 2：标注位置修复

任务目标：解决识别标签被图片边界裁切的问题。

现象：当手掌靠近图片顶部时，标签不可见。

任务：

1. 找到 `draw_info_text()` 函数。
2. 分析原代码为什么把文字画到了图片外。
3. 修改标签绘制逻辑：上方空间不够时，将标签画在边界框下方。
4. 重新生成 `result.jpg`。结果图片中必须清楚显示手势标签。

小实验 3：检测阈值敏感性实验

任务目标：理解检测置信度对识别结果的影响。

分别运行：

```
python app.py --image hand.jpg --output result_03.jpg --no_display --
min_detection_confidence 0.3
python app.py --image hand.jpg --output result_07.jpg --no_display --
min_detection_confidence 0.7
python app.py --image hand.jpg --output result_09.jpg --no_display --
min_detection_confidence 0.9
```

记录结果：

阈值	是否检测到手	识别标签	观察
0.3			
0.7			
0.9			

思考问题：

- 阈值过低可能带来什么问题？
- 阈值过高为什么可能导致检测不到手？
- 在物联网设备中，应该如何选择阈值？

小实验 4：输入图片质量实验

任务目标：分析真实应用中图像质量对识别的影响。

准备 4 类图片：

图片类型	说明
正常图片	手部清晰，背景简单
模糊图片	运动模糊或压缩严重
遮挡图片	手指被遮挡一部分
复杂背景图片	背景颜色和手部接近，或背景中有干扰物

分别识别后记录：

```
python app.py --image test1.jpg --output test1_result.jpg --no_display
python app.py --image test2.jpg --output test2_result.jpg --no_display
python app.py --image test3.jpg --output test3_result.jpg --no_display
python app.py --image test4.jpg --output test4_result.jpg --no_display
```

小实验 5：批量识别脚本

任务目标：将单张图片识别扩展成批量处理任务。

建议新建文件：

```
batch_run.ps1
```

参考 PowerShell 命令：

```
New-Item -ItemType Directory -Force outputs
Get-Childitem .\inputs -Filter *.jpg | ForEach-Object {
    python app.py --image $_.FullName --output (Join-Path "outputs" ($_.BaseName
+ "_result.jpg")) --no_display
}
```

任务：

1. 新建 inputs/ 文件夹，放入多张手势图片。
2. 执行批量识别。
3. 检查 outputs/ 文件夹中的结果。

思考问题：

- 批量识别时如何统计每类手势出现次数？
- 如果要把识别结果上传到服务器，应该上传原图还是上传标签？

小实验 6：IoT 控制规则设计

任务目标：把识别结果映射为物联网控制动作。

示例规则：

手势	控制动作
Open	打开灯光
Close	关闭灯光
Pointer	下一页
OK	确认操作

任务：

在 `run_image_mode()` 中，根据 `hand_sign_text` 增加控制输出，例如：

```
if hand_sign_text == 'Open':
    print('IoT Action: light_on')
elif hand_sign_text == 'Close':
    print('IoT Action: light_off')
elif hand_sign_text == 'Pointer':
    print('IoT Action: next_page')
elif hand_sign_text == 'OK':
    print('IoT Action: confirm')
```

进阶要求：

- 将动作写入 `action_log.csv`。
- 设计一个 MQTT 消息格式，例如：

```
{"device_id": "edge_camera_01", "gesture": "Open", "action": "light_on"}
```

小实验 7：新增或调整手势标签

任务目标：了解模型标签文件和训练数据之间的关系。

观察文件：

```
model/keypoint_classifier/keypoint_classifier_label.csv
```

当前内容：

```
Open
Close
Pointer
OK
```

注意：仅修改标签文件不会改变模型能力，只会改变输出文字。真正增加新手势需要：

1. 采集新手势关键点数据。

2. 修改训练 Notebook 中的类别数。
3. 重新训练模型。
4. 导出新的 TFLite 模型。
5. 更新标签文件。

七、实验报告要求

实验报告建议包含以下内容：

7.1 基础部分

- 实验环境：Python 版本、主要依赖版本。
- 运行命令：识别 `hand.jpg` 的命令。
- 运行结果：终端输出和 `result.jpg` 截图。
- 简述流程：图片如何经过 MediaPipe 和 TFLite 得到手势标签。

7.2 拓展部分

至少完成拓展任务中的 2 个。

八、参考资料

1. MediaPipe Hands: https://developers.google.com/mediapipe/solutions/vision/hand_landmarker
2. TensorFlow Lite: <https://www.tensorflow.org/lite>
3. OpenCV Python: <https://docs.opencv.org/>
4. 原始项目: <https://github.com/Kazuhito00/hand-gesture-recognition-using-mediapipe>